

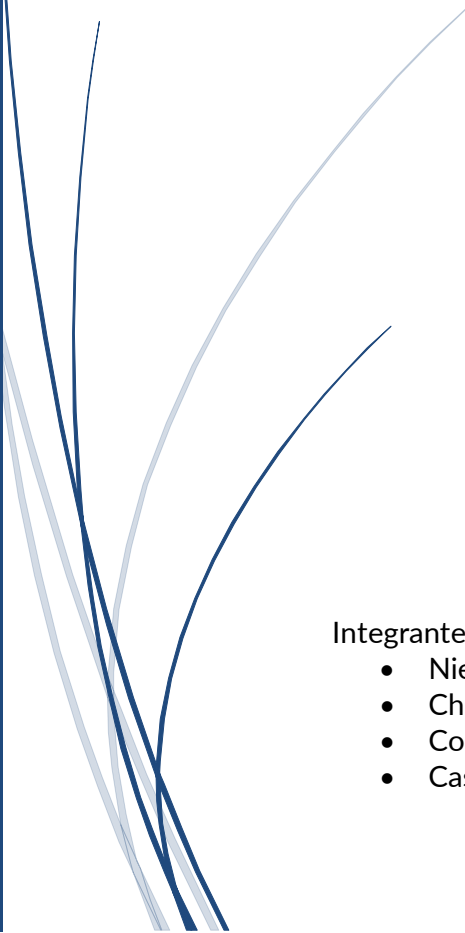


COLEGIO
UNIVERSITARIO



Aplicaciones web II

Analista de Sistemas y Desarrollo de
Software



Integrantes:

- Nievas Rocío
- Chanquia Antonella
- Cordoba Milagros
- Casagrande Maria Trinidad

Índice

.....	0
Proyecto Web – “SANTIAGO CEREALES SRL”	2
1. Introducción	2
2. Equipo de trabajo y roles.....	2
3. Descripción del proyecto	3
4. Justificación	4
5. Objetivos	4
6. Cronograma de trabajo	4
7. Herramienta de gestión del proyecto	5
8. Herramienta de control de versiones	5
9. Repositorio.....	5
10. Herramienta de seguimiento	6
11. Conclusión	6
12. Avance TP2 — Back-end para servir el sitio web	6

Proyecto Web – “SANTIAGO CEREALES SRL”

TP1 – Propuesta de Desarrollo del Proyecto

1. Introducción

El presente documento tiene como objetivo presentar la propuesta de desarrollo del proyecto que se llevará a cabo durante la materia Aplicaciones Web II.

Como base del trabajo se utilizará el proyecto **Santiago Cereales SRL**, desarrollado previamente en la materia Aplicaciones Web I. Durante esta nueva etapa se buscará evolucionar dicho proyecto incorporando funcionalidades del lado del servidor (Back-End), manteniendo el Front-End previamente desarrollado y ampliando sus capacidades.

El propósito es continuar el desarrollo de una aplicación web que pueda convertirse en una solución funcional para la empresa, permitiendo administrar la información de manera más eficiente y sentando las bases para una futura implementación real.

Visión del proyecto

El proyecto busca transformar el sitio web institucional desarrollado en Aplicaciones Web I en una aplicación web completa, incorporando progresivamente las tecnologías y conocimientos adquiridos durante la materia Aplicaciones Web II. Además del objetivo académico, se pretende desarrollar una solución que pueda ser presentada como propuesta real para la empresa Santiago Cereales SRL

2. Equipo de trabajo y roles

El proyecto será desarrollado por un equipo de cuatro integrantes. Cada miembro tendrá un rol principal para facilitar la organización del trabajo, aunque las responsabilidades podrán rotarse a lo largo del cuatrimestre con el objetivo de que todos los integrantes adquieran experiencia en las distintas áreas del desarrollo Back-End.

Integrante	Rol principal	Responsabilidades
Rocío Nievas	Líder del proyecto y Desarrollo Back-End	Coordinación del equipo, planificación, desarrollo de la lógica del servidor, integración con el Front-End y seguimiento de GitHub Projects.
Antonella Chanquia	Desarrollo Back-End	Implementación de funcionalidades, colaboración en el desarrollo del servidor, documentación técnica y control de versiones.
Milagros Córdoba	Base de Datos y API	Diseño y administración de la base de datos, creación de modelos, integración con la API y realización de pruebas de funcionamiento.
Maria Trinidad Casagrande	Testing, Documentación y Calidad	Pruebas del sistema, detección de errores, validación de funcionalidades, actualización de la documentación y colaboración en las entregas.

3. Descripción del proyecto

El proyecto consiste en continuar el desarrollo del sitio web **Santiago Cereales SRL**, realizado en la materia Aplicaciones Web I, incorporando nuevas funcionalidades correspondientes al desarrollo Back-End.

En la primera etapa se desarrolló un sitio web institucional que permite a los usuarios conocer la empresa, sus servicios, productos y medios de contacto.

Durante Aplicaciones Web II se buscará ampliar dicho proyecto incorporando nuevas funcionalidades que permitan gestionar la información del sistema de manera organizada, manteniendo el sitio desarrollado previamente como interfaz para los usuarios.

De esta forma, el proyecto evolucionará progresivamente durante la cursada hasta transformarse en una aplicación web más completa.

El desarrollo del Back-End se llevará a cabo utilizando Node.js y el framework Express, que permiten crear un servidor propio encargado de publicar el sitio y de exponer rutas (endpoints) propias. Estas rutas consultan los datos a través de una API externa (MockAPI), evitando que la información quede almacenada en archivos estáticos como ocurría en la etapa anterior. A lo largo de la cursada, este servidor irá incorporando progresivamente nuevas capacidades: en primer lugar la conexión con la API externa (TP2), luego operaciones de creación, lectura, actualización y eliminación de datos —CRUD— (TP3), y finalmente un sistema de autenticación y seguridad para proteger dichas operaciones (TP4).

4. Justificación

Actualmente el sitio web desarrollado presenta la información de forma estática, siendo necesario modificar el código fuente para realizar cualquier actualización.

La continuidad del proyecto permitirá incorporar herramientas que faciliten la administración de la información y mejoren la organización del sistema.

Además del objetivo académico, el proyecto busca generar una solución que pueda adaptarse a una necesidad real de la empresa Santiago Cereales SRL, constituyendo una base para una futura implementación.

5. Objetivos

Objetivo general

Continuar el desarrollo del proyecto Santiago Cereales SRL incorporando las funcionalidades correspondientes a la materia Aplicaciones Web II.

Objetivos específicos

- Continuar utilizando el Front-End desarrollado en Aplicaciones Web I.
- Incorporar progresivamente nuevas funcionalidades durante la cursada.
- Mantener una estructura ordenada del proyecto.
- Aplicar buenas prácticas de desarrollo y documentación.
- Gestionar correctamente las versiones del proyecto.
- Implementar un servidor propio utilizando Node.js y Express.
- Conectar el sistema a una API externa para el manejo de los datos, evitando la dependencia de archivos estáticos.
- Incorporar progresivamente operaciones CRUD y mecanismos de autenticación en las siguientes etapas del proyecto.

6. Cronograma de trabajo

Entrega	Actividad	Fecha límite
---------	-----------	--------------

TP1	Propuesta de desarrollo	28/08/2026
TP2	Implementación del servidor para Front-End	18/09/2026
TP3	Desarrollo del CRUD	16/10/2026
TP4	Desarrollo del Login y seguridad del CRUD	06/11/2026

Las actividades se desarrollarán respetando el cronograma establecido por la cátedra.

7. Herramienta de gestión del proyecto

Para la organización del proyecto se utilizará **GitHub Projects**.

Esta herramienta permitirá planificar las tareas, asignar actividades, realizar el seguimiento del avance del proyecto y mantener un control organizado de cada una de las etapas de desarrollo.

Las tareas serán clasificadas en distintas columnas según su estado de avance, facilitando el trabajo colaborativo entre ambos integrantes.

8. Herramienta de control de versiones

Se utilizará **Git** como sistema de control de versiones y **GitHub** como plataforma para alojar el repositorio del proyecto.

El uso de Git permitirá registrar el historial de modificaciones realizadas durante el desarrollo, facilitando el trabajo colaborativo y el seguimiento de los avances mediante commits realizados dentro de las fechas establecidas por la cátedra.

9. Repositorio

El repositorio contendrá:

- Código fuente del proyecto.
- Documento PDF correspondiente al TP1.
- Archivos del proyecto.
- Historial de commits.
- Documentación necesaria para el desarrollo.

10. Herramienta de seguimiento

Se utilizará **GitHub Projects** para gestionar las tareas del proyecto.

El tablero permitirá visualizar el estado de cada actividad mediante columnas de trabajo como:

- Pendiente
 - En desarrollo
 - En revisión
 - Finalizado
-

11. Conclusión

El presente proyecto representa la continuidad del trabajo realizado en Aplicaciones Web I, permitiendo evolucionar el sitio web desarrollado hacia una aplicación con mayores funcionalidades.

Durante el transcurso de la materia se incorporarán progresivamente nuevos conocimientos y herramientas propias del desarrollo Back-End, manteniendo una metodología de trabajo organizada mediante el uso de Git, GitHub y GitHub Projects.

El objetivo final es obtener una aplicación web funcional, escalable y bien documentada, que además de cumplir con los requisitos académicos, pueda convertirse en una propuesta de valor para una futura implementación en la empresa Santiago Cereales SRL.

12. Avance TP2 — Back-end para servir el sitio web

Este apartado documenta el desarrollo correspondiente al TP2, cuyo objetivo fue transformar el sitio estático desarrollado en Aplicaciones Web I en una aplicación servida por un back-end propio, que obtiene sus datos desde una API externa en lugar de un archivo JSON local.

Tecnologías utilizadas

- **Node.js:** entorno de ejecución de JavaScript del lado del servidor.
- **Express.js:** framework utilizado para crear el servidor y definir las rutas (endpoints).
- **ES Modules (.mjs, import/export):** sistema de módulos utilizado para organizar el código en archivos separados.
- **MockAPI:** plataforma utilizada para simular una API externa, donde se cargaron los datos de categorías y productos de la empresa.

Arquitectura implementada

El Front-End no se conecta directamente a MockAPI. Se implementó un servidor propio que actúa como intermediario entre el Front-End y la fuente de datos externa:

```
Front-End (HTML / CSS / JS)
  |
  v
Servidor propio (Node.js + Express)
  |
  v
MockAPI (datos externos)
```

- Sirve los archivos estáticos del Front-End desarrollado en AW1, mediante `express.static`.
- Expone rutas propias que consultan los datos en MockAPI y se los devuelven al Front-End ya procesados.

Middlewares implementados

Un middleware es una función que se ejecuta entre la llegada de una petición al servidor y la generación de la respuesta. Recibe los objetos `req` (petición) y `res` (respuesta), más una función `next()` que debe invocar para que la petición continúe su camino hacia la siguiente función o ruta. Todos los middlewares se registran mediante `app.use()`. En el proyecto se utilizan los siguientes:

- **`express.static("."): middleware`** incluido en Express que sirve los archivos estáticos del Front-End (HTML, CSS, JS, imágenes) directamente desde la raíz del proyecto.
- **`productosRouter:`** el conjunto de rutas propias (`/categorias`, `/productos/:categoriald`, `/productos`) se organiza en un Router de Express y se registra como middleware en `server.mjs`.

- **express.json():** middleware incluido en Express que procesa el cuerpo (body) de las peticiones que llegan en formato JSON y lo deja disponible en req.body. Es necesario para poder leer los datos enviados por método POST.
- **logger (propio):** middleware simple, escrito para este proyecto, que registra en la consola del servidor el método HTTP y la ruta de cada petición recibida (por ejemplo, POST /productos), a modo de evidencia del funcionamiento del servidor.

```
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next();
}

app.use(logger);
app.use(express.json());
app.use(express.static("."));
app.use(productosRouter);
```

Modelo de datos

Se definieron dos recursos relacionados en MockAPI:

- **categorias:** las 5 categorías de productos de la empresa (Herbicidas, Fungicidas, Fertilizantes, Semillas, Insecticidas), cada una con nombre e imagen.
- **productos:** productos individuales, cada uno con nombre, marca, descripción, imagen y categoriaId (referencia a la categoría a la que pertenece).

Estructura de carpetas del proyecto

```
/ (raíz del proyecto)
server.mjs           -> configura y levanta el servidor
/routes
  productos.mjs      -> define los endpoints propios
/css, /js, /recursos -> Front-End existente (sin cambios de
organización)
index.html, empresa.html, contacto.html, serviciosyproductos.html
package.json
```

Endpoints desarrollados

Método	Ruta	Descripción
GET	(base-url)/categorias	Obtiene todas las categorías de productos disponibles.
GET	(base-url)/productos/:categoriaId	Obtiene los productos pertenecientes a una categoría específica, identificada por su ID.

POST	<code>(base-url)/productos</code>	Crea un nuevo producto: recibe sus datos en el cuerpo de la petición (JSON) y los envía a MockAPI para almacenarlos.
-------------	-----------------------------------	--

Integración con el Front-End existente

Se modificó `js/productos.js` para que, en lugar de leer `productos.json` de forma local, consuma los endpoints propios del servidor. Al hacer click en una categoría, se solicitan sus productos (con marca y descripción) al segundo endpoint y se muestran dinámicamente en pantalla.

El endpoint `POST /productos`, en cambio, no se consume todavía desde una pantalla del Front-End: la carga de productos nuevos forma parte del CRUD que se desarrollará en el TP3. Por el momento se probó de forma manual con la extensión Thunder Client, enviando una petición `POST` con un producto en formato JSON y verificando en la consola del servidor y en MockAPI que se creaba correctamente.

Dificultades encontradas y resueltas

- Diferencia de tipos de datos entre el parámetro recibido en la URL (siempre texto) y el campo `categoriald` almacenado como número en MockAPI, que impedía que el filtro encontrara coincidencias. Se resolvió convirtiendo el valor con `Number()` antes de comparar.
- Error de referencia de archivos al ejecutar el servidor, por tener `server.mjs` ubicado en una carpeta distinta a la raíz del proyecto.